

RC1800842

Project brief — TCN2508

Project Title: Automated Offensive & Defensive Cybersecurity Framework — *OffDef Suite*

Overview (one line):

Design and implement a modular Python-based framework that automates safe reconnaissance, vulnerability discovery, exploitation *in a controlled lab*, post-exploit analysis and command/control simulation — **plus integrated defensive modules** (IDS, log analysis, automated countermeasures). The goal is to teach end-to-end red/blue workflows, tool integration, and secure responsible disclosure practices.

Key principles (non-negotiable)

- **Ethical & legal scope:** All testing **MUST** run only in isolated, consented lab environments (VMs, containers, private networks, intentionally vulnerable targets). No scanning or exploitation of third-party systems. Implement a signed project agreement.
- **Safety-first design:** Defensive modules must be central — detection, alerting, automated containment.
- **Modularity & reusability:** Each capability is a discrete module with clean interfaces so teams can swap components and reuse in other projects.
- **Reproducible delivery:** Code, docs, and VMs/images will be versioned and published to the team GitHub repo with clear runbooks.

Learning outcomes

Students will demonstrate ability to:

1. Build automated offensive toolchains (recon → vuln scan → exploitation → post-exploit).
2. Implement detection logic (IDS signatures, SIEM ingestion) and automated responses (firewall rules, isolate hosts).
3. Integrate telemetry with a logging/visualisation stack (Wazuh/Splunk/ELK).
4. Produce professional deliverables: README, architecture diagrams, test plans, ethics & disclosure statement, demo video.
5. Map attacks to MITRE ATT&CK and produce mitigation guidance.

RC1800842

Technical stack (recommended)

- **Language:** Python 3.10+ (modules, packages).
- **Networking:** Scapy, sockets, paramiko (SSH, with ethical use), requests.
- **Containers & VMs:** Docker for components; target VMs in VirtualBox/VMware or local lab images (Metasploitable, DVWA, VulnHub).
- **Detection & logging:** Wazuh agent / Filebeat → Elasticsearch / OpenSearch → Kibana (or Splunk if available).
- **Orchestration & CI:** GitHub (repo + Actions for lint/tests), Docker Compose for local orchestration.
- **Reporting:** HTML/PDF generator (Markdown → PDF), CSV/JSON export for findings.
- **Documentation:** MkDocs or GitHub Pages for project site.

Project architecture (high-level)

1. **Core controller (CLI/daemon)** — orchestrates modules, schedules scans, authenticates operators, generates reports.
2. **Recon module** — passive + active enumeration (DNS, ports, basic service banners). Rate-limited and lab-only.
3. **Vuln mapping module** — queries local/whitelisted CVE DB; matches service versions to CVEs.
4. **Exploitation simulator** — safe, parameterized exploit runners for lab targets (no internet exploitation). Uses local proof-of-concepts in isolated environment.
5. **Post-exploit module** — limited artifact collection, credential capture (lab only), persistence simulation; emphasise forensics traceability.
6. **C2 simulator** — toy C2 to demonstrate command execution and detection; used only in lab.
7. **IDS & detection module** — signature and anomaly detection (Snort/Suricata + custom rules).
8. **Log analyzer & SIEM integration** — collects telemetry, performs correlation, triggers alerts.
9. **Countermeasure engine** — auto-apply firewall rules, disable network interface on sim host, or notify SOC playbook triggers.
10. **Reporting & mitigation guide** — automatic actionable report with remediation steps & MITRE mappings.

RC1800842

Lab integration mapping (to course labs)

- **Lab 1 — Variables / Basic Functions:** build small network utilities (IP parser, simple port check).
- **Lab 2 — Functions & Loops:** multithreaded scanner, brute-force loop patterns (lab only).
- **Lab 3 — Slicing/Casting:** parse and normalize logs, extract fields for SIEM.
- **Lab 4 — Network Services:** ARP spoofing demo in a controlled VM network; packet capture analysis.
- **Lab 5 — I/O:** log collectors & report generation files.
- **Lab 6 — Modules:** packaging code as modular Python packages
- **Lab 7 — Advanced functions:** C2 simulation, automated playbooks.

Deliverables (per team)

1. **GitHub repo** with:
 - Code (well-commented), `requirements.txt`,
 - Unit tests and linters configured (pre-commit)
 - README with quickstart and safety warnings
2. **Architecture diagram** (PNG + editable source)
3. **Runbook & lab setup guide** (how to build local lab safely)
4. **Final report** (PDF): methodology, results, MITRE mapping, remediation
5. **Demo video** (5–8 minutes) showing safe lab run
6. **Presentation** (10–12 slides)
7. **Ethics & Responsible Disclosure statement** included in repo
8. **Instructor checkpoint notes** and peer review log

RC1800842

Assessment rubric (example)

- **Functionality (30%)** — modules perform intended tasks; safe execution in lab.
- **Security & Safety (20%)** — adherence to ethical rules, non-destructive testing, logging.
- **Detection & Mitigation (20%)** — quality of IDS rules, SIEM correlation, automated responses.
- **Code quality & docs (15%)** — tests, modularity, clear README, runbook.
- **Presentation & report (15%)** — clarity, mapping to MITRE, remediation guidance.

-).

Responsible disclosure & licensing

- Include `ETHICS.md` and `RULES_OF_ENGAGEMENT.md` in repo.
- Use permissive license (MIT) but clearly forbid malicious use; include a note that code is intended for authorized lab use only.
- Document responsible disclosure steps for any real vulnerabilities found in collaborative partner projects.

Final note to students

This project is a capstone: it must show technical depth **and** ethical responsibility. Focus on clean, testable modules and strong detection/mitigation work — employers value candidates who can both break and build resilient systems. Ask mentors early for lab approval and architectural feedback.

RC1800842

TCN2508 – PYTHON PROJECT: INDUSTRY DEVELOPMENT GROUP 1 Project

Title: Automated Cyber Attack & Defense Framework: Offensive and Defensive Security Suite

Project Overview:

This project integrates multiple penetration-testing techniques into a single, comprehensive cybersecurity framework. It automates reconnaissance, exploitation, post-exploitation, and defensive countermeasures to mitigate attacks. The suite includes modules for scanning, exploitation, privilege escalation, and a built-in reporting system.

Project Structure & Lab Integration

Phase 1 – Target Reconnaissance & Enumeration- Port Scanner (Lab 1 & Lab 2) – Python-based scanner that discovers open ports & services.

- Uses multithreading for speed.
- Vulnerability Scanner (Lab 3) – Queries external vulnerability databases (e.g., CVE lists) to match discovered services with known flaws.

Phase 2 – Exploitation & Credential Attacks- SSH Brute-Force Attack (Lab 4) – Dictionary attack on SSH services.

- ARP Spoofing & Packet Sniffing (Lab 5 & Lab 6) – Capture network traffic using ARP spoofing.
- Keylogger Integration (Lab 6 & Lab 7) – Stealth keylogger for credential harvesting.

Phase 3 – Post-Exploitation & Command & Control (C2)- C2 Server (Lab 7) – Remote access module to control compromised machines, execute shell commands, and maintain persistence.

Phase 4 – Web Security & Injection Attacks- SQL Injection (Lab 3 & Lab 6) – Automated extraction of database data.

- Command Injection & Authentication Bypass (Lab 5 & Lab 7) – Exploits insecure authentication mechanisms.

RC1800842

- CORS & SSRF Exploits (Lab 6) – Simulates misconfigurations leading to data exfiltration.

Phase 5 – Defensive Security Measures- Intrusion Detection System (IDS) – Detects scanning & brute-force attempts.

- Defensive Log Analyzer – Monitors security incidents in real-time.
- Countermeasures – Automatic firewall rule updates, anti-keylogger techniques, and hardening scripts.

Final Deliverables-

- ✓ Unified Python-based framework integrating all attack & defense modules.
- ✓ Automated attack execution & detailed reporting system.
- ✓ Defensive security features that mitigate discovered attacks.
- ✓ Comprehensive documentation & final project report.
- ✓ Source code hosted on GitHub and professional portfolio platforms.

Access & Visibility-

GitHub Repository: Create Your Git-Hub Repository and Upload the codes there

- Online Portfolio: Create your LinkedIn Profile and have your Video Uploaded
- Create Pentest Platform: Hackerone, Yeswehack, Integriti, Bugcrowd and show active registration engagement

Project Significance Provides real-world, hands-on cybersecurity experience, preparing participants for penetration testing, red-team, and defensive security roles. 🚀🔒